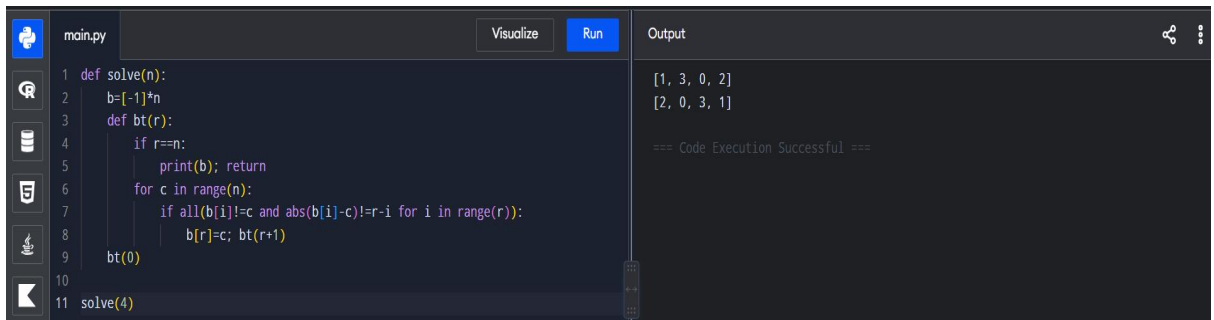


TOPIC 6: BACKTRACKING

1. N-Queens



```
main.py Visualize Run
```

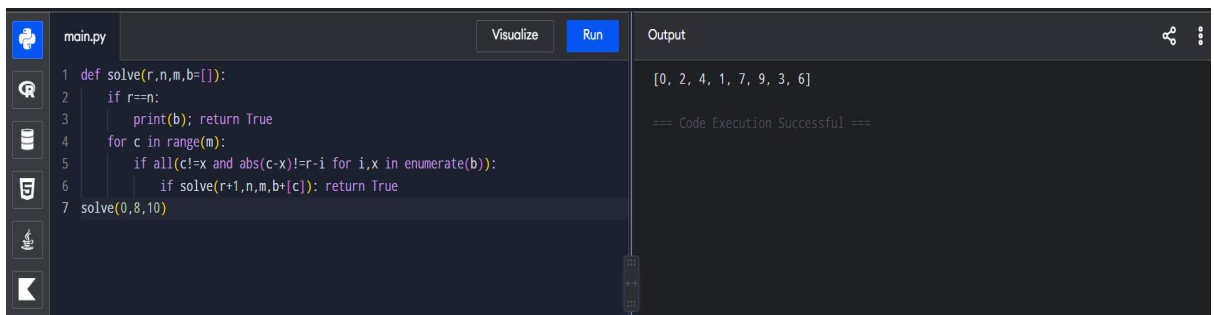
```
1 def solve(n):
2     b=[-1]*n
3     def bt(r):
4         if r==n:
5             print(b); return
6         for c in range(n):
7             if all(b[i]!=c and abs(b[i]-c)!=r-i for i in range(r)):
8                 b[r]=c; bt(r+1)
9     bt(0)
10
11 solve(4)
```

```
Output
```

```
[1, 3, 0, 2]
[2, 0, 3, 1]

=== Code Execution Successful ===
```

2. Generalized N-Queens — 8×10



```
main.py Visualize Run
```

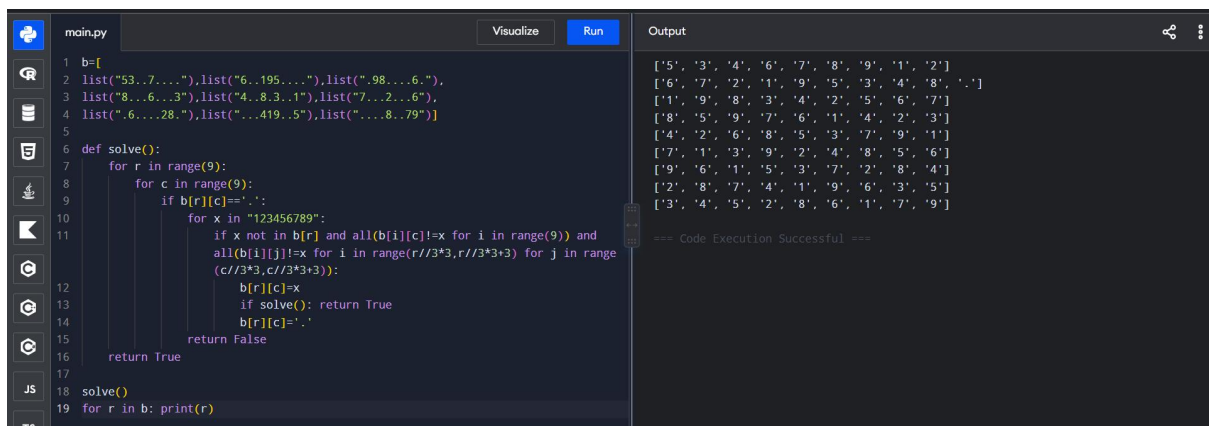
```
1 def solve(r,n,m,b=[]):
2     if r==n:
3         print(b); return True
4     for c in range(m):
5         if all(c!=x and abs(c-x)!=r-i for i,x in enumerate(b)):
6             if solve(r+1,n,m,b+[c]): return True
7 solve(0,8,10)
```

```
Output
```

```
[0, 2, 4, 1, 7, 9, 3, 6]

=== Code Execution Successful ===
```

3. Sudoku



```
main.py Visualize Run
```

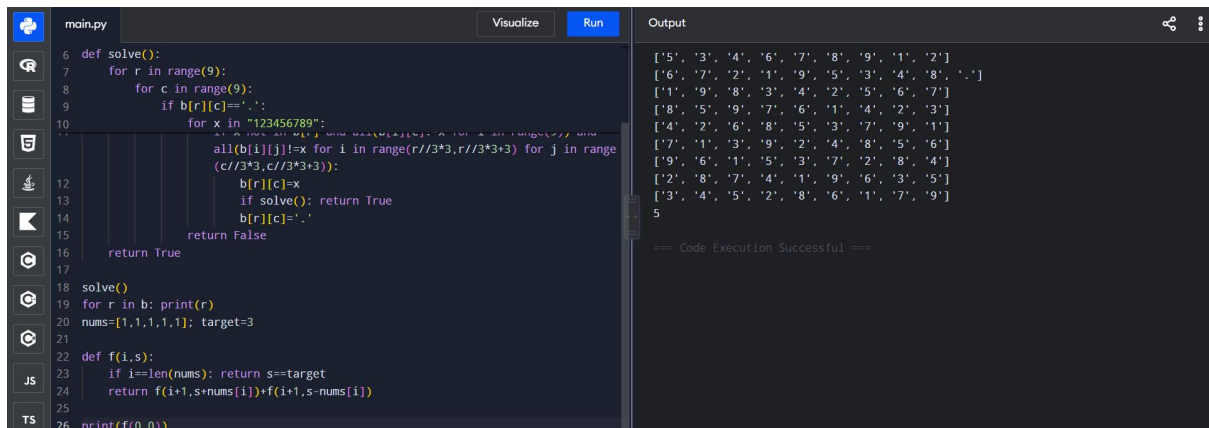
```
1 b=[
2     list("53..7..."),list("6..195...."),list("..98....6."),
3     list("8...6...3"),list("4..83..1"),list("7...2...6"),
4     list("..6....28."),list("...419...5"),list("....8..79")]
5
6 def solve():
7     for r in range(9):
8         for c in range(9):
9             if b[r][c]=='.':
10                 for x in "123456789":
11                     if x not in b[r] and all(b[i][c]!=x for i in range(9)) and
12                        all(b[i][j]!=x for i in range(r//3*3,r//3*3+3) for j in range(c//3*3,c//3*3+3)):
13                         b[r][c]=x
14                         if solve(): return True
15                         b[r][c]='.'
16     return False
17
18 solve()
19 for r in b: print(r)
```

```
Output
```

```
[['5', '3', '4', '6', '7', '8', '9', '1', '2'],
 ['6', '7', '2', '1', '9', '5', '3', '4', '8'],
 ['1', '9', '8', '3', '4', '2', '5', '6', '7'],
 ['8', '5', '9', '7', '6', '1', '4', '2', '3'],
 ['4', '2', '6', '8', '5', '3', '7', '9', '1'],
 ['7', '1', '3', '9', '2', '4', '8', '5', '6'],
 ['9', '6', '1', '5', '3', '7', '2', '8', '4'],
 ['2', '8', '7', '4', '1', '9', '6', '3', '5'],
 ['3', '4', '5', '2', '8', '6', '1', '7', '9']]

=== Code Execution Successful ===
```

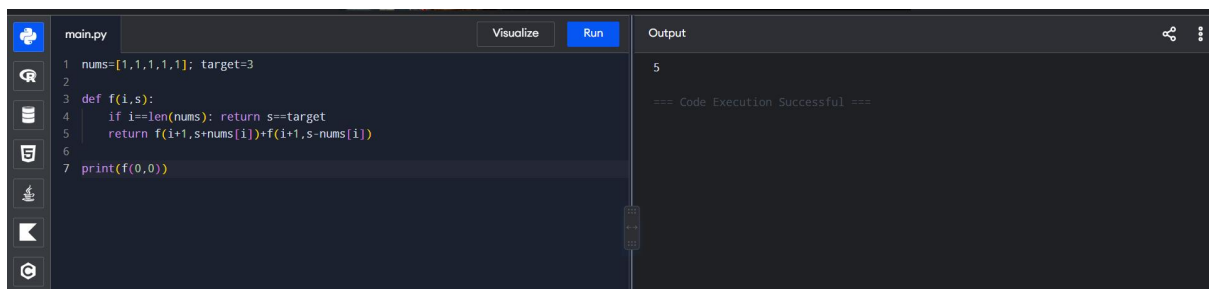
4. Sudoku



```
main.py Visualize Run Output
6 def solve():
7     for r in range(9):
8         for c in range(9):
9             if b[r][c]!='.':
10                for x in "123456789":
11                    if not all(b[i][j]!=x for i in range(r//3*3,r//3*3+3) for j in range(c//3*3,c//3*3+3)):
12                        b[r][c]=x
13                        if solve(): return True
14                        b[r][c]='.'
15                return False
16        return True
17
18 solve()
19 for r in b: print(r)
20 nums=[1,1,1,1,1]; target=3
21
22 def f(i,s):
23     if i==len(nums): return s==target
24     return f(i+1,s+nums[i])+f(i+1,s-nums[i])
25
26 print(f(0,0))
```

```
['5', '3', '4', '6', '7', '8', '9', '1', '2']
['6', '7', '2', '1', '9', '5', '3', '4', '8', '.']
['1', '9', '8', '3', '4', '2', '5', '6', '7']
['8', '5', '9', '7', '6', '1', '4', '2', '3']
['4', '2', '6', '8', '5', '3', '7', '9', '1']
['7', '1', '3', '9', '2', '4', '8', '5', '6']
['9', '6', '1', '5', '3', '7', '2', '8', '4']
['2', '8', '7', '4', '1', '9', '6', '3', '5']
['3', '4', '5', '2', '8', '6', '1', '7', '9']
5
=== Code Execution Successful ===
```

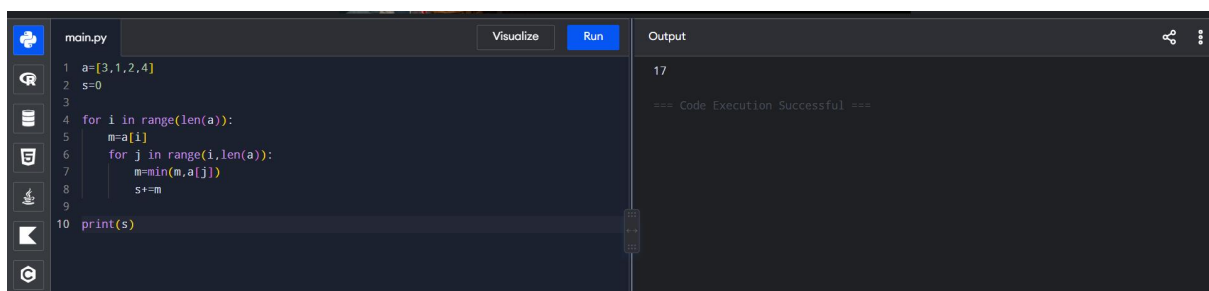
5. Target Sum



```
main.py Visualize Run Output
1 nums=[1,1,1,1,1]; target=3
2
3 def f(i,s):
4     if i==len(nums): return s==target
5     return f(i+1,s+nums[i])+f(i+1,s-nums[i])
6
7 print(f(0,0))
```

```
5
=== Code Execution Successful ===
```

6. Sum of Subarray Minimums



```
main.py Visualize Run Output
1 a=[3,1,2,4]
2 s=0
3
4 for i in range(len(a)):
5     m=a[i]
6     for j in range(i,len(a)):
7         m=min(m,a[j])
8         s+=m
9
10 print(s)
```

```
17
=== Code Execution Successful ===
```

7. Combination Sum

```
main.py Visualize Run Output
1 a=[2,3,6,7]; t=7
2 ans=[]
3
4 def f(i,t,x):
5     if t==0: ans.append(x); return
6     for j in range(i,len(a)):
7         if a[j]<=t: f(j,t-a[j],x+[a[j]])
8
9 f(0,t,[])
10 print(ans)
```

[[2, 2, 3], [7]]

=== Code Execution Successful ===

8. Combination Sum II

```
main.py Visualize Run Output
1 a=[10,1,2,7,6,1,5]; t=8
2 a.sort(); ans=[]
3
4 def f(i,t,x):
5     if t==0: ans.append(x); return
6     for j in range(i,len(a)):
7         if j>i and a[j]==a[j-1]: continue
8         if a[j]>t: break
9         f(j+1,t-a[j],x+[a[j]])
10
11 f(0,t,[])
12 print(ans)
```

[[1, 1, 6], [1, 2, 5], [1, 7], [2, 6]]

=== Code Execution Successful ===

9. Permutations

```
main.py Visualize Run Output
1 a=[1,2,3]; ans=[]
2
3 def f(x,r):
4     if not r: ans.append(x); return
5     for v in r: f(x+[v],r-{v})
6
7 f([],set(a))
8 print(ans)
```

[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]

=== Code Execution Successful ===

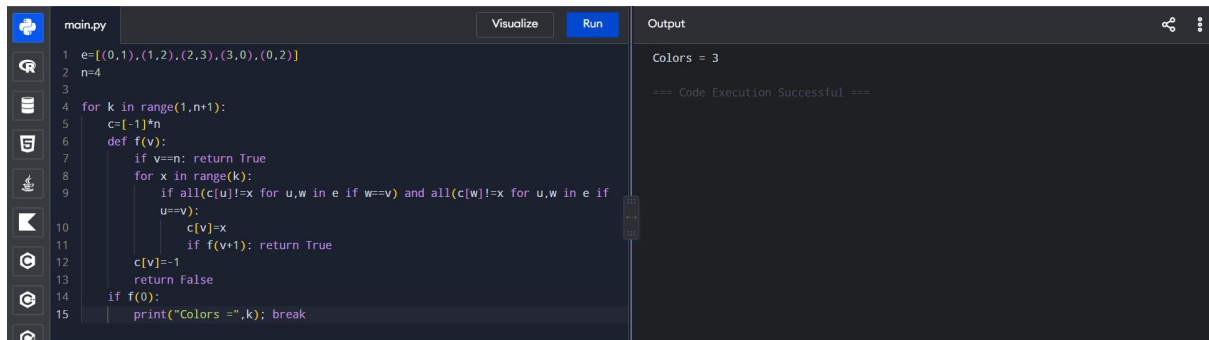
10. Unique Permutations

```
main.py Visualize Run Output
1 from itertools import permutations
2
3 a = [1,1,2]
4 print(list(set(permutations(a))))
```

[(1, 2, 1), (2, 1, 1), (1, 1, 2)]

=== Code Execution Successful ===

11. Graph Coloring — Minimum Colors

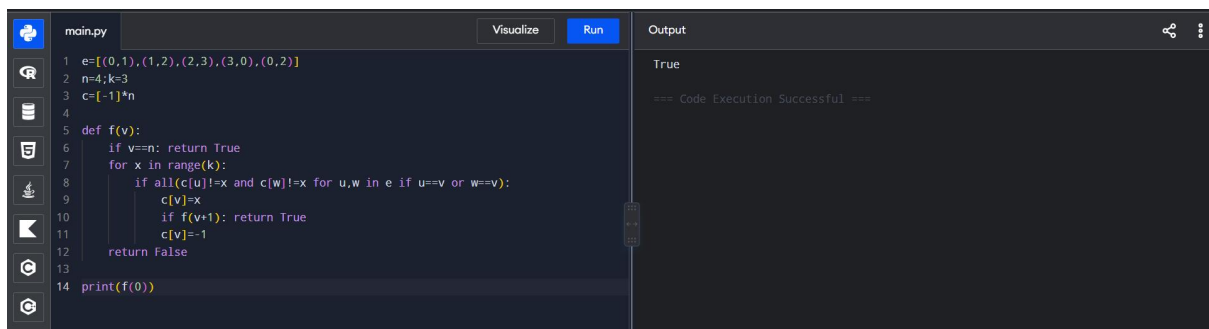


```
main.py Visualize Run Output
1 e=[(0,1),(1,2),(2,3),(3,0),(0,2)]
2 n=4
3
4 for k in range(1,n+1):
5     c=[-1]*n
6     def f(v):
7         if v==n: return True
8         for x in range(k):
9             if all(c[u]!=x for u,w in e if w==v) and all(c[w]!=x for u,w in e if
10                u==v):
11                 c[v]=x
12                 if f(v+1): return True
13             c[v]=-1
14             return False
15     if f(0):
16         print("Colors =",k); break
```

Colors = 3

=== Code Execution Successful ===

12. Graph Coloring — k = 3

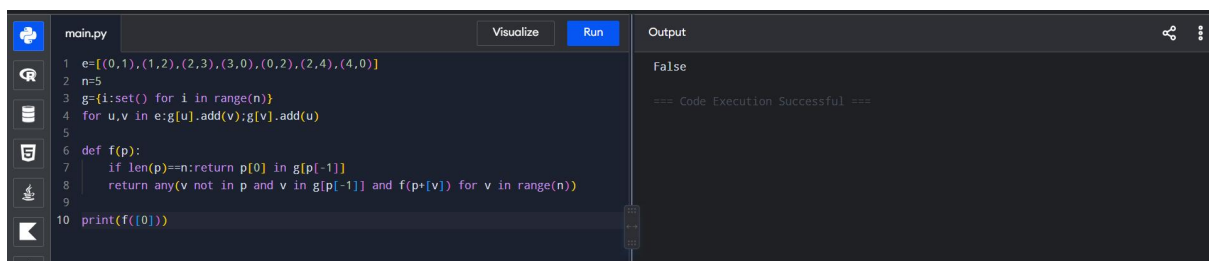


```
main.py Visualize Run Output
1 e=[(0,1),(1,2),(2,3),(3,0),(0,2)]
2 n=4;k=3
3 c=[-1]*n
4
5 def f(v):
6     if v==n: return True
7     for x in range(k):
8         if all(c[u]!=x and c[w]!=x for u,w in e if u==v or w==v):
9             c[v]=x
10            if f(v+1): return True
11            c[v]=-1
12            return False
13
14 print(f(0))
```

True

=== Code Execution Successful ===

13. Hamiltonian Cycle

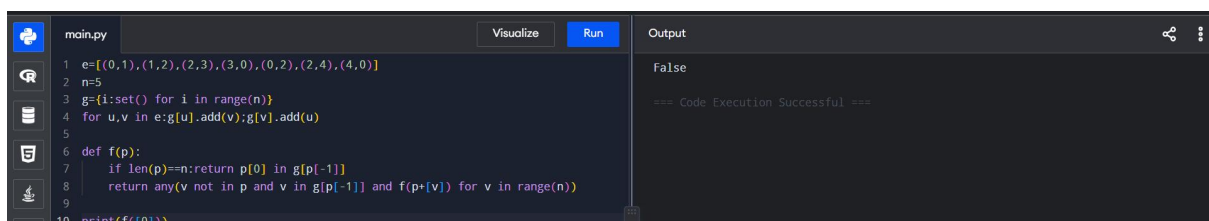


```
main.py Visualize Run Output
1 e=[(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
2 n=5
3 g={i:set() for i in range(n)}
4 for u,v in e:g[u].add(v);g[v].add(u)
5
6 def f(p):
7     if len(p)==n: return p[0] in g[p[-1]]
8     return any(v not in p and v in g[p[-1]] and f(p+[v]) for v in range(n))
9
10 print(f([0]))
```

False

=== Code Execution Successful ===

14. Hamiltonian Cycle

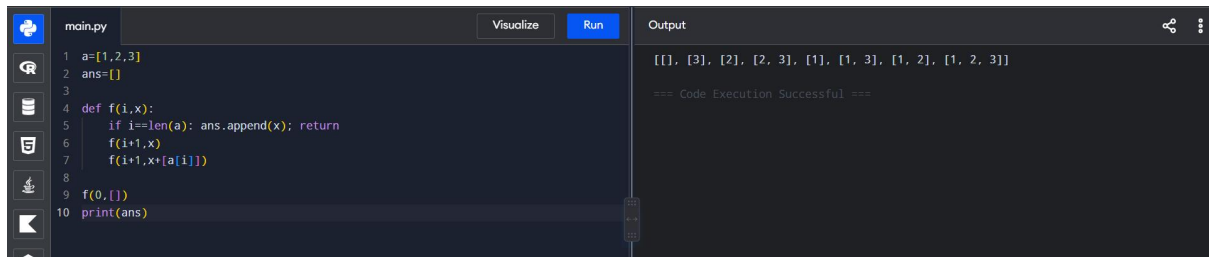


```
main.py Visualize Run Output
1 e=[(0,1),(1,2),(2,3),(3,0),(0,2),(2,4),(4,0)]
2 n=5
3 g={i:set() for i in range(n)}
4 for u,v in e:g[u].add(v);g[v].add(u)
5
6 def f(p):
7     if len(p)==n: return p[0] in g[p[-1]]
8     return any(v not in p and v in g[p[-1]] and f(p+[v]) for v in range(n))
9
10 print(f([0]))
```

False

=== Code Execution Successful ===

15. Generate All Subsets

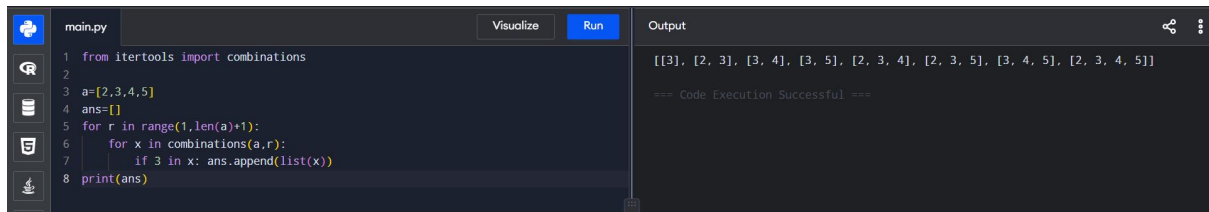


```
main.py Visualize Run Output
1 a=[1,2,3]
2 ans=[]
3
4 def f(i,x):
5     if i==len(a): ans.append(x); return
6     f(i+1,x)
7     f(i+1,x+a[i])
8
9 f(0,[])
10 print(ans)
```

[[], [3], [2], [2, 3], [1], [1, 3], [1, 2], [1, 2, 3]]

=== Code Execution Successful ===

16. Subsets Containing 3

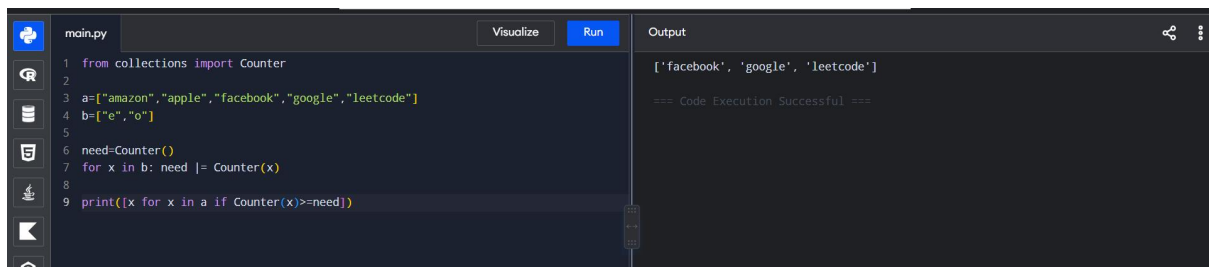


```
main.py Visualize Run Output
1 from itertools import combinations
2
3 a=[2,3,4,5]
4 ans=[]
5 for r in range(1,len(a)+1):
6     for x in combinations(a,r):
7         if 3 in x: ans.append(list(x))
8 print(ans)
```

[[3], [2, 3], [3, 4], [3, 5], [2, 3, 4], [2, 3, 5], [3, 4, 5], [2, 3, 4, 5]]

=== Code Execution Successful ===

17. Universal Strings



```
main.py Visualize Run Output
1 from collections import Counter
2
3 a=["amazon","apple","facebook","google","leetcode"]
4 b=["e","o"]
5
6 need=Counter()
7 for x in b: need |= Counter(x)
8
9 print([x for x in a if Counter(x)>=need])
```

['facebook', 'google', 'leetcode']

=== Code Execution Successful ===